

挑战大模型：算法正确性与复杂度审计

算法正确性与复杂度审计

姓名：张祺昀 学号：23300200015 日期：2026年9月21日

摘要

本文通过两个具体问题审计大模型的算法推理。任务一考察确定性RTS战斗中的集火策略：在双方单位初始属性完全相同的条件下，最低虚拟生命值优先的贪心策略使己方全灭，而另一条合法策略能保留5点生命值。反例揭示了局部伤害分配与跨时刻最优决策的区别，并以完整动作枚举和记忆化搜索给出修正。任务二考察按 $n/\log n$ 与剩余部分划分的递归算法。模型错误地给出 $\Theta(n \log n / \log \log n)$ ，本文定位其势函数差分错误，并证明正确复杂度为 $\Theta(n(\log n)^2 / \log \log n)$ 。两项审计均以数学论证为核心，以独立程序复核为辅助。

一、任务一：RTS集火算法的正确性审计

1.1 问题定义

输入为己方数量 n 、敌方数量 m 、共同目标点 G ，以及双方初始坐标。每方1至3个单位，编号分别为 $F_1, \dots, F_n$ 和 $E_1, \dots, E_m$ ；所有输入坐标分量属于 $[-30, 30]$ ，且为0.5的整数倍。单位是可重叠的点，沿当前位置到 G 的直线移动，不越过 G 。没有障碍、碰撞、技能、随机伤害或治疗。

共同属性	取值	共同属性	取值
初始生命值	55	攻击伤害 / 护甲	10 / 0
射程（含边界）	6	每tick最大移动距离	2
攻击间隔	2 tick	初始状态	全部Ready

己方Ready单位可攻击射程内任意存活敌人，也可等待；等待保留Ready。敌方Ready且有合法目标时必须攻击最近的己方单位，距离相同取编号最小者。第 t 个tick出手后，最早在 $t+2$ 再次攻击。目标是最大化终局己方存活单位总生命值，并输出一条达到该值的合法决策序列。允许指数级精确算法。

任务一模型信息为 gpt-5.6-luna、最低推理档位

1.2 战斗语义与被审计的贪心策略

每个tick按如下顺序执行：①在tick开始状态上确定双方全部攻击；②累加伤害并同时扣血，再移除死亡单位，已经确定的攻击不因攻击者当场死亡而取消；③任一方全灭即结束，同时全灭的己方收益为0；④存活攻击者的下一tick冷却设为1，其他冷却减1且不低于0；⑤基于死亡移除后、任何移动前的统一位置快照，同时执行所有符合条件的移动。

可移动单位须本tick未攻击、尚未到G，且射程内没有存活敌人，每次前进 $\min(2, \text{到G距离})$ 。射程内有敌人时，即使正在冷却或主动等待，也不得移动；移动后首次进入射程，只能从下一tick判断攻击。

回答整理稿采用“最低虚拟生命值优先”。每个tick复制敌方生命值，按己方编号分配攻击；排序键依次为虚拟生命值、真实生命值、敌方编号。下面是统一属性下的等价伪代码：

```
FocusFire(s):
    virtualHP = copy(enemyHP); action = empty map
    for f in aliveFriendlyById:
        if not Ready(f): action[f] = wait; continue
        C = alive enemies in range with virtualHP > 0
        if C is empty: action[f] = wait; continue
        e = argmin(C, (virtualHP[e], HP[e], id[e]))
        action[f] = attack(e)
        virtualHP[e] = max(0, virtualHP[e] - 10)
    return action
```

整理稿的最优性论证可概括为：优先集火低血量目标能更早完成击杀；敌人越少，未来伤害越低；重复此选择就能最大化己方终局生命值。虚拟生命值能减少当轮过量伤害，但这一局部性质尚不能推出全局最优。单次目标选择时间为 $O(nm)$ ，额外空间为 $O(n+m)$ 。

1.3 反例输入与共同开局

令 $G=(0,0)$ ， $F1=(-10,0)$ 、 $F2=(-7,0)$ 、 $E1=(3,0)$ 、 $E2=(4.5,0)$ 。四个单位均采用前页共同属性。以下HP顺序固定为 $(F1, F2, E1, E2)$ ，死亡记0；位置均为该tick移动后的x坐标。

tick	全部攻击	结算后HP	移动后x坐标
0	无	(55, 55, 55, 55)	(-8, -5, 1, 2.5)
1	$F2 \rightarrow E1$; $E1 \rightarrow F2$	(55, 45, 45, 55)	(-6, -5, 1, 0.5)
2	$E2 \rightarrow F2$	(55, 35, 45, 55)	(-4, -5, 1, 0.5)

tick 3开始时，两名己方单位都可攻击E1、E2；E1为45血且Ready，E2为55血且在冷却。随后位置不变，E1在奇数tick攻击，E2在偶数tick攻击。

1.4 完整反例轨迹：相同首次击杀时间，不同终局

旧贪心在tick 3优先攻击E1。对照策略则从tick 3起优先集火E2，击杀后再打E1。表中列出双方全部攻击和结算后HP；两条策略的tick 0—2均与前页一致。

tick	旧贪心：全部攻击	旧贪心：HP	对照策略：全部攻击	对照策略：HP
3	F1→E1; F2→E1; E1→F1	(45, 35, 25, 55)	F1→E2; F2→E2; E1→F1	(45, 35, 45, 35)
4	E2→F1	(35, 35, 25, 55)	E2→F1	(35, 35, 45, 35)
5	F1→E1; F2→E1; E1→F1	(25, 35, 5, 55)	F1→E2; F2→E2; E1→F1	(25, 35, 45, 15)
6	E2→F1	(15, 35, 5, 55)	E2→F1	(15, 35, 45, 15)
7	F1→E1; F2→E2; E1→F1	(5, 35, 0, 45)	F1→E2; F2→E2; E1→F1	(5, 35, 45, 0)
8	E2→F1	(0, 35, 0, 45)	无	(5, 35, 45, 0)
9	F2→E2	(0, 35, 0, 35)	F1→E1; F2→E1; E1→F1	(0, 35, 25, 0)
10	E2→F2	(0, 25, 0, 35)	无	(0, 35, 25, 0)
11	F2→E2	(0, 25, 0, 25)	F2→E1; E1→F2	(0, 25, 15, 0)
12	E2→F2	(0, 15, 0, 25)	无	(0, 25, 15, 0)
13	F2→E2	(0, 15, 0, 15)	F2→E1; E1→F2	(0, 15, 5, 0)
14	E2→F2	(0, 5, 0, 15)	无	(0, 15, 5, 0)
15	F2→E2	(0, 5, 0, 5)	F2→E1; E1→F2	(0, 5, 0, 0)
16	E2→F2	(0, 0, 0, 5)	已结束	(0, 5, 0, 0)

结果：旧贪心在tick 16己方全灭，收益为0；对照策略在tick 15消灭全部敌人，F2剩余5血，收益为5。只要存在这一条收益更高的合法策略，就足以反驳旧贪心的全局最优性，无需依赖穷举求出真正最优值。

两条策略都在tick 7完成首次击杀。关键差异是：击杀E1不能取消它在tick 7已经确定的攻击；击杀E2却能消除其tick 8攻击，使F1活到tick 9，多打出一10点伤害。

1.5 错误定位与人工修正

错误在于把“当前虚拟生命值最低”作为唯一目标，而没有证明该选择能延伸为整场战斗的最优方案。当前血量和第一次击杀时刻都不足以刻画未来收益；敌方的冷却相位、下一次攻击对象，以及被保护单位能否多攻击一次，都会改变结果。

这一错误具有迷惑性：统一初始属性掩盖了不同接敌时间造成的冷却差异；虚拟生命值确实能减少部分过量伤害，容易被扩大解释为全局正确；“敌人越少，伤害越低”忽略了敌人何时出手、攻击谁。基础场景中的成功也不能代替贪心选择性质的证明。

状态与动作

保留战斗转移规则，改为枚举所有合法联合动作。固定初始输入和单位编号，对每个存活单位记录生命值 H 、实际移动次数 q 和剩余冷却 $r \in \{0, 1\}$ ，死亡单位用统一哨兵表示。编号不能随意交换，因为敌方距离并列时会读取编号。

设初始位置为 p_0 ，到 G 距离为 d ， $L = \lceil d/2 \rceil$ 。若 $d > 0$ ，第 q 次移动后位置由下式唯一确定； $d=0$ 时始终位于 G 。因此状态键可以使用整数而不依赖浮点坐标哈希。

$$p(q) = p_0 + \min(1, 2q/d)(G - p_0).$$

每个 Ready 单位可选择任意合法目标或 wait，非 Ready 单位只有 wait。枚举这些选项的笛卡尔积，包括可能造成过量伤害的攻击；攻击与等待会改变冷却和移动状态，不能仅按虚拟血量提前删去分支。旧贪心动作可用于搜索排序，但不用于剪枝。

```
Solve(s):
    if Terminal(s): return SumFriendlyHP(s)
    k = Encode(s)
    if k in memo: return memo[k]
    best = -infinity; seen = empty set
    for a in AllLegalJointActions(s):
        next = SimulateOneTick(s, a)
        nk = Encode(next)
        if nk in seen: continue
        add nk to seen
        value = Solve(next)
        if value > best:
            best = value; choice[k] = a
    memo[k] = best
    return best
```

求解前将 memo、choice 初始化为空表，且仅在同一初始输入内使用。SimulateOneTick 严格执行第 1.2 节的战斗规则。由初始状态反复读取 choice 即可恢复最优序列。

1.6 终止性、最优性与复杂度

状态有限。生命值只减不增，存活时仅有55、45、35、25、15、5六种值；冷却两种，移动次数0至L。含死亡哨兵在内，单位i最多有 $1+12(L_i+1)$ 种状态。

严格下降。以死亡单位生命值为0，定义按字典序比较的非负整数三元组：

$$K = \sum_i \lfloor H_i/10 \rfloor, M = \sum_{i \text{ alive}} (L_i - q_i), C = \sum_{i \text{ alive}} r_i.$$

有攻击时，至少一个结算前存活的目标受伤，K严格下降；无攻击但有移动时，K不变而M严格下降；两者都没有时，双方必有单位接敌，否则未到G的单位应移动，而全到G也必接敌。此时敌方未出手说明其正在冷却，因此C严格下降。故每个非终局转移都进入更小状态，递归必终止。

最优性。状态包含决定后续行动与转移所需的全部信息。终局返回实际收益；对非终局，完整枚举所有联合动作，敌方行为确定，每个动作对应唯一后继。对上述良基顺序归纳：

$$V(s) = \max_{a \in A(s)} V(T(s,a)).$$

所有更小状态的最优值正确时，对全部后继取最大就得到当前状态的最优值。相同后继去重不改变可取得的结果集合。因此初始返回值为全局最优，choice恢复的序列合法且达到该值。

复杂度。记 $N=n+m$ ，可达状态数S，单状态最大联合动作数B，最大递归深度D。每次转移中的距离与范围检查为 $O(N^2)$ 。在单位代价几何运算和哈希表平均常数查询模型下：

$$B \leq (m+1)^n, S \leq \prod_{i=1 \dots N} [1+12(L_i+1)].$$

$$\text{Time} = O(SBN^2), \text{Space} = O(SN + DBN).$$

每条轨迹发生攻击的tick至多6N个，含移动的tick至多 $\sum L_i$ 个；纯冷却等待每次至多一个，总数不超过6N，因此 $D \leq 12N + \sum L_i$ 。空间上界包括记忆表、递归栈及各层后继去重集合。算法允许指数规模搜索；达到程序状态预算时明确报错，不把截断结果当作最优值。

验证场景	就近攻击	旧贪心	精确修正版
同属性反例	5	0	5
基础对称场景	0	25	25

反例记忆化搜索访问684个非终止状态，独立半整数坐标BFS访问737个状态，均确认最优值为5。两者状态表示与计数口径不同，状态数无需一致。

二、任务二：可变比例递归的复杂度审计

2.1 算法与实验设置

输入 n 为正整数。采用单位代价RAM模型，整数运算、比较、赋值、 $\text{floor}(\log_2 n)$ 和 $\text{work}()$ 均记为常数时间。整数不溢出，递归不使用记忆化，各次调用实际执行。分析变量为数值 n 。

```
procedure C(n):
  if n <= 3:
    work()
    return
  k = floor(log2(n))
  r = floor(n / k)
  C(r)
  C(n - r)
  for i = 1 to n:
    work()
```

$n \geq 4$ 时， $2 \leq k \leq n$ ，因此 $1 \leq r \leq n/2$ ，两个子问题均严格缩小，递归必终止。非递归工作为 $\Theta(n)$ ，要求解：

$$T(n) = T(r) + T(n - r) + \Theta(n), r = \lfloor n / \lfloor \log_2 n \rfloor \rfloor.$$

以gpt-5.6-luna的low（轻度思考）设置开启无历史上下文的独立实验，要求给出紧确 Θ 界和推导，实际提示词没有提供正确答案。

2.2 模型分析与被反驳的结论

模型正确建立了递推，随后选择 $F(n) = n \log n / \log \log n$ 作为候选，并令 $p = r/n = \Theta(1/\log n)$ 。下面摘录其关键等式与结论，公式仅作排版转换：

$$\begin{aligned} & F(n) - F(r) - F(n - r) \\ &= \Theta((n \log n / \log \log n) [p \log(1/p) + (1-p) \log(1/(1-p))]). \end{aligned}$$

“由于 $p \log(1/p) = \Theta(\log \log n / \log n)$ ，上式为 $\Theta(n)$ 。因此通过上下界归纳，递推解满足”

$$T(n) = \Theta(F(n)) = \Theta(n \log n / \log \log n).$$

原文还指出“取整和停止条件只影响低阶项，不改变该渐近界”。本报告反驳的是这一明确的紧确结论，以及支撑它的差分等式。

2.3 错误定位：势函数值与变化率混淆

以下用自然对数分析同阶候选函数：保持原文2底对数也得到相同的差分阶。令 $x=\ln n$ 、 $p=r/n=\Theta(1/x)$ ， $F(n)=n g(x)$ ， $g(x)=x/\ln x$ 。实际差分为：

$$\Delta_F = r[g(x)-g(\ln r)] + (n-r)[g(x)-g(\ln(n-r))].$$

由于 $\ln r=x-\Theta(\ln x)$ 、 $\ln(n-r)=x-\Theta(1/x)$ ，当 n 充分大时，区间内 u 与 x 同阶。函数 g 的导数为：

$$g'(u) = (\ln u - 1)/(\ln u)^2 = \Theta(1/\ln x).$$

定义加权对数下降量 $H(p)$ 。其两项分别为 $\Theta(\ln x/x)$ 和 $\Theta(1/x)$ ，故：

$$H(p) = p \ln(1/p) + (1-p) \ln(1/(1-p)) = \Theta(\ln x/x).$$

分别对两个差值应用中值定理，得到：

$$\Delta_F = \Theta(nH(p)/\ln x) = \Theta(n/\ln n).$$

错误恰在模型的“利用熵差”一步。它使用了 $\Theta(n g(x)H(p))$ ，实际应使用变化率 g' ，即 $\Theta(n g'(x)H(p))$ 。因此它把单次下降量多估了一个 $\ln n$ 因子。真实下降量不能抵偿每个节点的 $\Theta(n)$ 成本，“上下界归纳”的依据不成立。

2.4 不依赖精确解的下界反驳

递归树是满二叉树。各叶子规模在1到3之间，且规模之和为 n ，故叶子数 $L \geq n/3$ 。任意具有 L 个叶子的二叉树，深度小于 $\log_2(L/2)$ 的叶子至多 $L/2$ 个，所以叶子深度之和为 $\Omega(L \log L)$ 。

每个内部节点的工作量等于其规模。把规模分摊到后代叶子，总内部work次数为 Σ （叶子规模 $\times$ 叶子深度），至少为叶子深度之和。因此：

$$T(n) = \Omega(n \log n), n \log n / \log \log n = o(n \log n).$$

模型结论连这一普适下界都不满足，因而是实质性的复杂度错误，不只是证明过程略去细节。

2.5 正确紧确界的完整推导

仍令 $x=\ln n$ 、 $p=r/n$ 。由取整定义， p 被两个正常数倍的 $1/x$ 夹住。因此 $H(p)=\Theta(\ln x/x)$ 。这里 H 只是确定性的加权表达，不假设递归含有随机选择。

构造每次下降 $\Theta(n)$ 的势函数

$$G(x) = x^2/\ln x, \Phi(n) = nG(\ln n) = n(\ln n)^2/\ln \ln n.$$

在区间 $[\ln r, x]$ 内， $\ln r=x-\Theta(\ln x)$ ，故 $u=\Theta(x)$ 、 $\ln u=\Theta(\ln x)$ 。其导数一致满足：

$$G'(u) = u(2 \ln u - 1)/(\ln u)^2 = \Theta(x/\ln x).$$

对两个非负差值分别使用中值定理，单节点势能差为：

$$\begin{aligned}\Delta_\Phi &= \Phi(n) - \Phi(r) - \Phi(n-r) \\ &= \Theta((x/\ln x) [r \ln(n/r) + (n-r) \ln(n/(n-r))]) \\ &= \Theta((x/\ln x) nH(p)) = \Theta(n).\end{aligned}$$

因此存在与 n 无关的正常数 a 、 b 及阈值 N_0 ，使所有 $n \geq N_0$ 均满足 $a n \leq \Delta \Phi \leq b n$ 。这一统一界把势能变化与局部实际成本对应起来。

沿整棵递归树求和

在子问题规模小于固定阈值 N_0 时截断递归树，记内部节点集合为 I ，边界集合为 B 。对有限小规模任意正有界延拓 Φ ，必要时增大 N_0 。分裂保持规模总和，边界规模之和为 n ，个数至多 n ；边界势能与继续求解的总成本均为 $O(n)$ 。中间项相消得到：

$$\sum_{v \in I} \Delta_\Phi(n_v) = \Phi(n) - \sum_{b \in B} \Phi(n_b) = \Phi(n) - O(n) = \Theta(\Phi(n)).$$

最后一步使用 $\Phi(n)/n$ 趋于无穷。由 $a n_v \leq \Delta \Phi(n_v) \leq b n_v$ 可知，内部节点规模之和为 $\Theta(\Phi(n))$ 。加入 $O(n)$ 的边界成本，得到：

$$T(n) = \Theta(n(\log n)^2/\log \log n).$$

证明对全部充分大的正整数成立。 $\text{floor}(\log_2 n) = \Theta(\ln n)$ ， $\text{floor}(n/k)$ 与 n/k 相差不足1，而 n/k 趋于无穷，故取整不改变上述比例和导数估计；常数基例只影响 $O(n)$ 边界成本。不能因子问题规模之和为 n 就直接类比归并排序，也不能以最长路径乘 n 代替整树求和。

三、实验复核与综合结论

任务二以 $W(n)$ 统计work次数： $n \leq 3$ 时 $W(n)=1$ ；否则 $W(n)=W(r)+W(n-r)+n$ 。每个递归节点至少执行一次work，其他控制成本被其常数倍界定，故 $T(n)=\Theta(W(n))$ 。动态规划仅用于计算精确计数，不改变原算法无记忆化的定义。

n	精确work次数 $W(n)$	$W(n)/Q(n)$
256	2,518	0.461060
4,096	77,347	0.470117
65,536	1,976,168	0.471155
1,048,576	45,754,263	0.471465

$Q(n)=n(\log_2 n)^2/\log_2(\log_2 n)$ 。直接执行 $n=1\cdots 512$ 的全部work，与独立动态规划计数逐项相同。归一化结果与理论结论相容，但有限数值不替代渐近证明。

任务一以合法反例否定贪心最优性，任务二以单次差分检查否定错误复杂度。两者共同说明：局部合理的规则或形式完整的证明，仍须检查其能否支撑全局结论。对任务一，应验证贪心选择是否保留最优解；对任务二，应验证候选势函数是否实际覆盖节点成本。